AHSANULLAH UNIVERSITY OF SCIENCE AND TECHNOLOGY

Department of Computer Science and Engineering

Project Source Code

Course Code: CSE 3118

Group No:07

Submitted to,

Md. Zahid Hossain & Md. Mustofa Ahmed

Submitted by,

Name	ID
Danial Hossain Dani	20230104058
Md.Toufikul Alam Yame	20230104070
Md. Salah Uddin Yousuf	20230104074

Date of Submission:12 April, 2026

Source Code

Arduino UNO:

```
#include <Wire.h> // Connecting with LCD
#include <LiquidCrystal_I2C.h> // Controlling the LCD
#include <SoftwareSerial.h> // Communication with ESP 32

int flame1 = 2, flame2 = 3, flame3 = 4;
int gas1 = 5, gas2 = 6, gas3 = 7;

int led = 8, buzzer = 9;

#define ESP_RX 10
#define ESP_TX 11
SoftwareSerial espSerial(ESP_RX, ESP_TX);

LiquidCrystal_I2C lcd(0x27, 20, 4);

bool msgSent = false; // varible for msg
unsigned long lastMsgTime = 0;
const unsigned long msgInterval = 60000; // 60 s

void setup() {
  Serial.begin(9600);
  espSerial.begin(9600);
  lcd.init();
  lcd.backlight();
```

```
  lcd.setCursor(0,0); lcd.print("Fire Safety System");
  delay(1500);

  pinMode(flame1, INPUT); pinMode(flame2, INPUT); pinMode(flame3, INPUT);
  pinMode(gas1, INPUT); pinMode(gas2, INPUT); pinMode(gas3, INPUT);
  pinMode(led, OUTPUT); pinMode(buzzer, OUTPUT);
  digitalWrite(led, LOW); digitalWrite(buzzer, LOW);
}

void loop() {
  int f1 = digitalRead(flame1);
  int f2 = digitalRead(flame2);
  int f3 = digitalRead(flame3);
  int g1 = digitalRead(gas1);
  int g2 = digitalRead(gas2);
  int g3 = digitalRead(gas3);

  bool fireBlocks[3] = {f1 == LOW, f2 == LOW, f3 == LOW};
  bool gasBlocks[3]  = {g1 == LOW, g2 == LOW, g3 == LOW};
  int fireCount = fireBlocks[0] + fireBlocks[1] + fireBlocks[2];
  bool anyGas = gasBlocks[0] || gasBlocks[1] || gasBlocks[2];

  // Prepare display strings (same as before)
  String line0 = "";
  if (fireCount > 0) {
    line0 += "Fire:";
```

```
  if (fireBlocks[0]) line0 += " A";
  if (fireBlocks[1]) line0 += " B";
  if (fireBlocks[2]) line0 += " C";
}
if (anyGas) {
  if (line0 != "") line0 += " ";
  line0 += "Gas:";
  if (gasBlocks[0]) line0 += " A";
  if (gasBlocks[1]) line0 += " B";
  if (gasBlocks[2]) line0 += " C";
}

String line3 = "";
if (fireCount == 1) {
  if (fireBlocks[0]) line3 = "Use stairs B&C";
  else if (fireBlocks[1]) line3 = "Use stairs A&C";
  else if (fireBlocks[2]) line3 = "Use stairs A&B";
} else if (fireCount == 2) {
  if (!fireBlocks[0]) line3 = "Use stairs A";
  else if (!fireBlocks[1]) line3 = "Use stairs B";
  else if (!fireBlocks[2]) line3 = "Use stairs C";
} else if (fireCount == 3) line3 = "RIP";
else if (anyGas) line3 = "Check area";

bool dangerDetected = (fireCount > 0 || anyGas);
```

```
if (dangerDetected) {
  digitalWrite(led, HIGH);
  digitalWrite(buzzer, HIGH);

  String smsText = "";
  if (fireCount > 0) {
    smsText += "Fire on:";
    if (fireBlocks[0]) smsText += " A";
    if (fireBlocks[1]) smsText += " B";
    if (fireBlocks[2]) smsText += " C";
    smsText += ". ";
    if (fireCount < 3) smsText += line3 + ".";
    else smsText += "No stairs available!";
  }
  if (anyGas) {
    smsText += " Gas leak on:";
    if (gasBlocks[0]) smsText += " A";
    if (gasBlocks[1]) smsText += " B";
    if (gasBlocks[2]) smsText += " C";
    smsText += ".";
  }

  // LCD update
  lcd.clear();
  lcd.setCursor(0,0); lcd.print(line0);
  lcd.setCursor(0,1);
```

```
  lcd.print("F1:"); lcd.print(fireBlocks[0] ? "Flame" : "No");
  lcd.print(" F2:"); lcd.print(fireBlocks[1] ? "Flame" : "No");
  lcd.print(" F3:"); lcd.print(fireBlocks[2] ? "Flame" : "No");
  lcd.setCursor(0,2);
  lcd.print("G1:"); lcd.print(gasBlocks[0] ? "Gas" : "OK");
  lcd.print(" G2:"); lcd.print(gasBlocks[1] ? "Gas" : "OK");
  lcd.print(" G3:"); lcd.print(gasBlocks[2] ? "Gas" : "OK");
  lcd.setCursor(0,3); lcd.print(line3);

  // send to ESP32 (replaces GSM)
  unsigned long currentTime = millis();
  if (!msgSent || currentTime - lastMsgTime >= msgInterval) {
    espSerial.println(smsText);   // <-- ESP32 will read this
    msgSent = true;
    lastMsgTime = currentTime;
  }

  Serial.println("ALERT: " + smsText);
} else {
  digitalWrite(led, LOW);
  digitalWrite(buzzer, LOW);
  msgSent = false;
  lcd.clear();
  lcd.setCursor(0,0); lcd.print("Status: Normal");
  lcd.setCursor(0,1); lcd.print("F1:No F2:No F3:No");
  lcd.setCursor(0,2); lcd.print("G1:OK G2:OK G3:OK");
```

```
  lcd.setCursor(0,3); lcd.print("ESP32 Link Active");
 }

 delay(500);
}
```

ESP :

```
#include <WiFi.h>
#include <HTTPClient.h>

const char* ssid = "lucas";
const char* password = "01838883395@@@";

String botToken = "";
String chatID  = "7";

HardwareSerial unoSerial(2); // RX2=GPIO16, TX2=GPIO17

void setup() {
  Serial.begin(115200);
  unoSerial.begin(9600, SERIAL_8N1, 16, 17);
  Serial.println("Connecting WiFi...");
```

```
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected!");
  sendTelegram("🚨 Fire System Online");
}

void loop() {
  if (unoSerial.available()) {
    String msg = unoSerial.readStringUntil('\n');
    msg.trim();
    if (msg.length() > 0) {
      sendTelegram(msg);
      Serial.println("Sent to Telegram: " + msg);
    }
  }
}

void sendTelegram(String msg) {
  if (WiFi.status() == WL_CONNECTED) {
```

```
    HTTPClient http;
    String url = "https://api.telegram.org/bot" + botToken +
                "/sendMessage?chat_id=" + chatID +
                "&text=" + msg;
    http.begin(url);
    http.GET();
    http.end();
  }
}
```